

LAPORAN AKHIR PROJECT BASED LEARNING (PBL)
Mata Kuliah Algoritma & Struktur Data (TPL2106)

Tahun Akademik 2025/2026

Dosen: Dr. Shelve Nidya Neyman

APLIKASI MANAJEMEN KEUANGAN BERBASIS TERMINAL
(DUID Banking CLI)

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)
Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Syahirah Yusriyah Putri	J0403251003
Muhamad Ilham Nurhakim	J0403251004
Ahmad Maulana Abimanyu	J0403251017

Program Studi Teknologi Rekayasa Perangkat Lunak
Sekolah Vokasi IPB University
2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan akhir Project-Based Learning (PBL) mata kuliah Algoritma & Struktur Data (TPL2106) ini dengan baik.

Laporan ini membahas perancangan dan implementasi aplikasi manajemen keuangan berbasis terminal yang kami beri nama DUID Banking CLI. Aplikasi ini dirancang untuk membantu pengguna mengelola saldo dan transaksi keuangan secara terstruktur menggunakan berbagai konsep algoritma dan struktur data yang telah kami pelajari selama perkuliahan.

Kami mengucapkan terima kasih yang sebesar-besarnya kepada:

- Ibu Dr. Shelve Nidya Neyman selaku dosen pengampu mata kuliah Algoritma & Struktur Data yang telah memberikan bimbingan dan arahan selama proses pembelajaran dan pengerjaan proyek ini.
- Seluruh rekan kelompok yang telah berkontribusi penuh dalam proses perancangan, implementasi, hingga penulisan laporan ini.
- Keluarga dan teman-teman yang memberikan dukungan moral selama pengerjaan proyek.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun demi penyempurnaan laporan dan pengembangan aplikasi ke depannya.

Semoga laporan ini dapat bermanfaat bagi pembaca dan dapat menjadi referensi dalam pengembangan aplikasi serupa.

Bogor, Juni 2026

Tim Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I. PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan Proyek	4
1.3 Ruang Lingkup.....	5
BAB II. ANALISIS DAN PERANCANGAN	6
2.1 Deskripsi Sistem.....	6
2.2 Struktur Data yang Digunakan	6
Alasan Pemilihan Struktur Data	7
2.3 Algoritma yang Digunakan	7
Searching (Pencarian)	7
Sorting (Pengurutan)	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program	9
BAB III. IMPLEMENTASI PROGRAM	11
3.1 Tampilan Menu Utama.....	11
3.2 Fitur Create (Setor & Transfer).....	11
3.3 Fitur Read (Mutasi Rekening).....	13
3.4 Fitur Delete (Hapus Transaksi Lama)	15
3.5 Fitur Searching	15
3.6 Fitur Sorting	16
3.7 Penyimpanan Data (File Handling).....	16
3.8 Validasi Input dan Error Handling	17
BAB IV. PENGUJIAN DAN ANALISIS.....	18
4.1 Skenario Pengujian.....	18
4.2 Kelebihan Sistem.....	19
4.3 Keterbatasan Sistem	20
BAB VI. PEMBAGIAN TUGAS ANGGOTA	22
BAB VII. KESIMPULAN DAN SARAN	23
Kesimpulan.....	23
Saran Pengembangan	23
DAFTAR PUSTAKA	25
LAMPIRAN	26

BAB I. PENDAHULUAN

1.1 Latar Belakang

Pengelolaan keuangan pribadi merupakan salah satu aspek penting dalam kehidupan sehari-hari. Namun, banyak orang kesulitan untuk mencatat dan memantau riwayat transaksi keuangan mereka secara terstruktur. Penggunaan aplikasi perbankan digital memang telah umum, namun tidak semua orang memiliki akses atau kemampuan teknis untuk menggunakannya. Di sisi lain, pencatatan manual menggunakan buku atau spreadsheet rentan terhadap kesalahan dan tidak efisien.

Berangkat dari permasalahan tersebut, kami merancang aplikasi DUID Banking CLI sebuah sistem manajemen keuangan berbasis terminal (Command Line Interface) yang memungkinkan pengguna melakukan berbagai operasi perbankan sederhana secara langsung melalui antarmuka teks. Aplikasi ini mampu menyimpan data secara permanen menggunakan file JSON, serta mengimplementasikan berbagai struktur data dan algoritma yang dipelajari dalam mata kuliah Algoritma & Struktur Data.

Dengan pendekatan berbasis CLI, aplikasi ini tidak memerlukan koneksi internet maupun antarmuka grafis, sehingga dapat berjalan di berbagai lingkungan komputasi dengan kebutuhan sumber daya yang minimal. Seluruh data transaksi disimpan secara lokal dalam format JSON yang terstruktur, memastikan data tetap aman dan dapat diakses kembali setiap kali program dijalankan.

1.2 Tujuan Proyek

Tujuan dari pengembangan aplikasi DUID Banking CLI ini adalah:

- Membuat aplikasi manajemen keuangan berbasis terminal menggunakan Python yang fungsional dan mudah digunakan.
- Mengimplementasikan struktur data Dictionary, List, dan defaultdict untuk menyimpan dan mengelola data transaksi secara efisien.
- Menerapkan konsep file handling menggunakan format JSON agar data transaksi tersimpan secara permanen dan dapat diakses kembali.
- Mengimplementasikan algoritma searching untuk fitur filter transaksi berdasarkan jenis, tanggal, dan nominal.
- Mengimplementasikan algoritma sorting untuk menampilkan transaksi berdasarkan urutan tanggal dan nominal.

- Mengembangkan solusi terhadap permasalahan pengelolaan keuangan pribadi yang nyata dengan pendekatan yang sederhana namun terstruktur.
- Menerapkan validasi input dan error handling yang komprehensif untuk memastikan integritas data.

1.3 Ruang Lingkup

Ruang lingkup aplikasi DUID Banking CLI meliputi:

- Platform: CLI (Console/Terminal) berbasis Python 3.
- Bahasa Pemrograman: Python 3 dengan pustaka standar (os, json, datetime, collections) dan ReportLab untuk export PDF.
- Penyimpanan Data: File JSON (riwayat_duid.json) untuk persistensi data saldo dan transaksi.
- Fitur CRUD: Create (setor tunai, transfer, virtual account), Read (tampilkan riwayat mutasi), Delete (hapus transaksi lama).
- Fitur Searching: Filter transaksi berdasarkan jenis transaksi menggunakan linear search.
- Fitur Sorting: Pengurutan transaksi berdasarkan tanggal (terbaru ke terlama) dan nominal (terbesar ke terkecil).
- Fitur Laporan: Ringkasan keuangan per bulan dan per bank/layanan.
- Fitur Export: Generate laporan transaksi dalam format PDF menggunakan pustaka ReportLab.
- Antarmuka Web (hp.py): Mini ATM berbasis Flask yang dapat diakses melalui browser HP untuk setor tunai secara mudah, berbagi file JSON yang sama dengan bank.py.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

DUID Banking CLI adalah aplikasi manajemen keuangan berbasis terminal yang dirancang untuk mensimulasikan fungsi perbankan dasar. Sistem ini memungkinkan pengguna untuk melakukan transfer uang ke rekening bank lain, pembayaran melalui virtual account e-wallet, setor tunai, serta memantau seluruh riwayat transaksi secara real-time.

Seluruh data termasuk saldo pengguna dan riwayat transaksi disimpan dalam satu file JSON (`riwayat_duid.json`) yang dibaca setiap kali program dijalankan dan diperbarui setelah setiap operasi yang mengubah data. Dengan pendekatan ini, data tetap tersedia meski program ditutup dan dibuka kembali.

Antarmuka pengguna dirancang dengan tampilan menu hierarkis yang intuitif: menu utama menampilkan saldo terkini dan pilihan operasi, sedangkan sub-menu menyediakan fungsi yang lebih spesifik. Sistem juga dilengkapi dengan fitur paginasi untuk menampilkan daftar transaksi yang panjang, sehingga pengguna tidak terbebani dengan informasi yang terlalu banyak sekaligus.

Selain antarmuka CLI, proyek ini dilengkapi dengan komponen web bernama **hp.py** yang dibangun menggunakan framework Flask. Komponen ini berfungsi sebagai “ATM Mini” yang dapat diakses melalui browser di perangkat HP dalam jaringan lokal yang sama. `hp.py` secara langsung membaca dan menulis ke file JSON yang sama (`riwayat_duid.json`) yang digunakan oleh `bank.py`, sehingga data saldo dan transaksi selalu tersinkronisasi antara kedua antarmuka tanpa diperlukan database tambahan.

2.2 Struktur Data yang Digunakan

Tabel berikut merangkum struktur data yang digunakan dalam aplikasi beserta fungsinya:

Struktur Data	Fungsi dan Penjelasan
Dictionary (dict)	Menyimpan data utama aplikasi: saldo (float) dan daftar transaksi (list). Setiap transaksi juga direpresentasikan sebagai dictionary dengan key: tanggal, nama, rek, nominal, bank, jenis. Dictionary dipilih karena memungkinkan akses data $O(1)$ berdasarkan key yang sudah diketahui.

List	Menyimpan koleksi transaksi secara berurutan (ordered). List mendukung operasi append untuk menambah transaksi baru di akhir, slicing untuk paginasi, dan sorting untuk pengurutan. Dipilih karena transaksi perlu diakses secara sequential dan diurutkan.
defaultdict (collections)	Digunakan dalam modul laporan keuangan untuk agregasi data per bulan dan per bank. defaultdict(lambda: {"masuk": 0.0, "keluar": 0.0}) secara otomatis menginisialisasi nilai default saat key baru diakses, menghindari KeyError dan menyederhanakan kode agregasi.
String	Digunakan untuk menyimpan data teks (nama, jenis transaksi, nomor rekening) dan untuk operasi pencarian/filter berbasis teks menggunakan metode .lower() dan in operator.

Alasan Pemilihan Struktur Data

Dictionary dipilih sebagai struktur data utama karena format JSON yang digunakan untuk persistensi data secara alami memetakan ke dictionary Python. Hal ini menyederhanakan operasi baca-tulis file karena tidak diperlukan konversi data yang kompleks json.load() menghasilkan dictionary dan json.dump() menerima dictionary secara langsung.

List dipilih untuk menyimpan koleksi transaksi karena urutan kronologis penting dalam konteks perbankan. List juga mendukung operasi sorting dan slicing yang dibutuhkan untuk fitur pengurutan dan paginasi dengan sintaks yang bersih dan efisien.

defaultdict dari modul collections dipilih untuk agregasi laporan karena menghilangkan kebutuhan untuk memeriksa keberadaan key sebelum menambahkan nilai, sehingga kode agregasi menjadi lebih ringkas dan mudah dibaca.

2.3 Algoritma yang Digunakan

Searching (Pencarian)

Aplikasi menggunakan Linear Search (Sequential Search) untuk fitur filter transaksi. Implementasinya menggunakan Python list comprehension yang secara internal melakukan traversal linear terhadap seluruh elemen list transaksi.

```
# Filter berdasarkan jenis transaksi
hasil = [t for t in transaksi if t['jenis'] == jenis]
```

```
# Filter pemasukan menggunakan keyword matching
def is_pemasukan(t: dict) -> bool:
    teks = (str(t.get('jenis', '')) + " " + str(t.get('bank',
    ''))).lower()
    kata_kunci = ['setor', 'tunai', 'cash', 'pemasukan', 'terima']
    return any(kata in teks for kata in kata_kunci)
```

Alasan penggunaan: Linear Search dipilih karena data transaksi tidak selalu dalam keadaan terurut berdasarkan jenis, dan jumlah data yang diproses (riwayat transaksi personal) tidak cukup besar untuk memerlukan algoritma yang lebih kompleks seperti Binary Search atau hashing. Kompleksitas waktu $O(n)$ masih sangat acceptable untuk use case ini.

Sorting (Pengurutan)

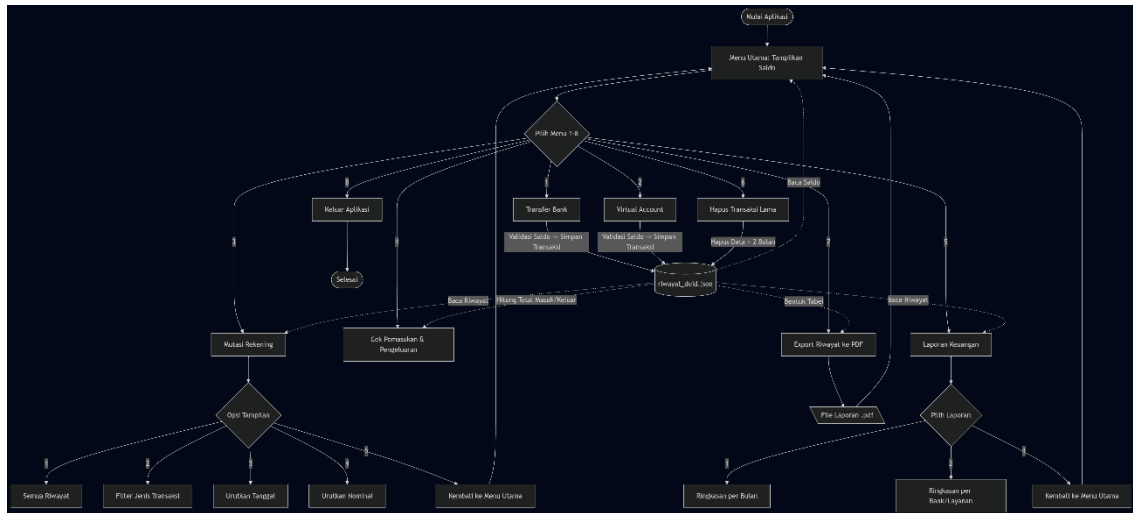
Aplikasi menggunakan algoritma Timsort melalui fungsi bawaan `sorted()` Python untuk mengurutkan transaksi. Timsort adalah algoritma hybrid yang menggabungkan Merge Sort dan Insertion Sort, dengan kompleksitas waktu rata-rata $O(n \log n)$.

```
# Sort berdasarkan tanggal (terbaru ke terlama)
urutan = sorted(transaksi, key=lambda x: x['tanggal'], reverse=True)

# Sort berdasarkan nominal (terbesar ke terkecil)
urutan = sorted(transaksi, key=lambda x: float(x['nominal']),
reverse=True)
```

Alasan penggunaan: Timsort dipilih karena merupakan algoritma sorting paling efisien yang tersedia secara built-in di Python dan sangat optimal untuk data yang sudah sebagian terurut (nearly-sorted data), yang umum terjadi pada riwayat transaksi. Key function menggunakan lambda untuk fleksibilitas dalam memilih field yang dijadikan kunci pengurutan.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Aplikasi terdiri dari dua file utama dengan organisasi sebagai berikut:

File / Modul	Deskripsi
bank.py	File utama CLI yang berisi seluruh logika aplikasi: fungsi I/O, validasi input, operasi transaksi (transfer, VA, setor), laporan keuangan, hapus transaksi lama, dan export PDF.
hp.py	Antarmuka web berbasis Flask (ATM Mini). Menyediakan form setor tunai yang dapat diakses dari browser HP melalui jaringan lokal (port 5000). Membaca dan menulis langsung ke riwayat_duid.json yang sama dengan bank.py sehingga data selalu tersinkronisasi.
riwayat_duid.json	File penyimpanan data (dibuat otomatis). Menyimpan saldo terkini dan seluruh riwayat transaksi dalam format JSON. Diakses bersama oleh bank.py dan hp.py.
laporan_transaksi_*.pdf	File laporan yang digenerate oleh fitur export PDF. Nama file menyertakan timestamp untuk keunikan.

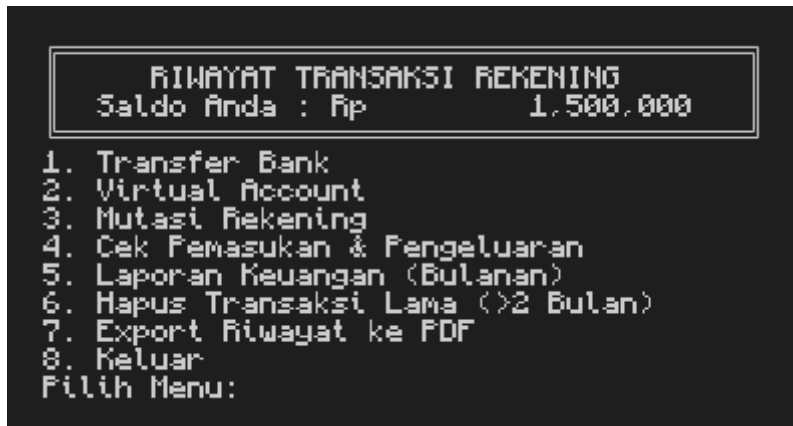
Organisasi fungsi dalam bank.py:

- Fungsi utilitas: `is_pemasukan()`, `baca_data()`, `simpan_data()`, `baca_saldo()`, `baca_transaksi()`
- Fungsi tampilan: `cetak_header()`, `cetak_baris()`, `cetak_pagination()`

- Fungsi validasi input: `input_nama()`, `input_nominal()`, `input_rek()`, `input_va()`, `konfirmasi()`
- Fungsi transaksi: `transfer_bank()`, `virtual_account()`, `setor_tunai()`
- Fungsi menu: `menu_tampilan()`, `menu_laporan()`, `cek_rekap_keuangan()`
- Fungsi utilitas lanjutan: `hapus_transaksi_lama()`, `export_pdf()`
- Fungsi utama: `main()`

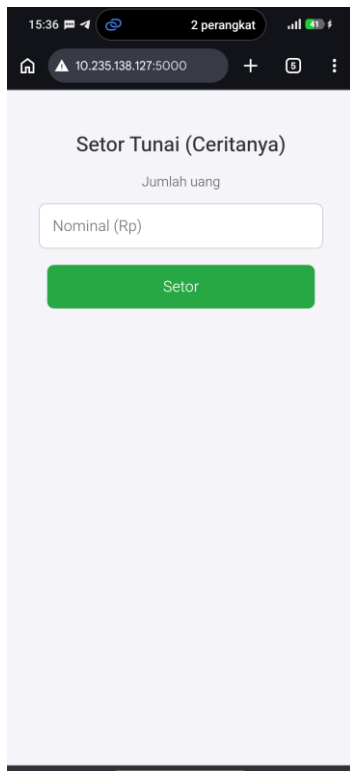
BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama



Menu utama aplikasi menampilkan informasi saldo terkini dan delapan pilihan operasi yang dapat dipilih pengguna. Tampilan menggunakan karakter ASCII art untuk membuat antarmuka lebih menarik secara visual meski berbasis teks. Saldo ditampilkan dengan format rupiah yang diformat menggunakan f-string dengan separator ribuan.

3.2 Fitur Create (Setor & Transfer)



```
=== TRANSFER BANK ===

Saldo tersedia : Rp 1.500.000
Nama pemilik rekening : Ironmen
Nomor rekening : 1234567890
Nama bank : BCA
Nominal (Rp) : 50000

Ringkasan Transaksi
Jenis : Transfer Bank
Nama : Ironmen
Rek : 1234567890
Bank : BCA
Nominal : Rp 50.000
Sisa Saldo: Rp 1.450.000

Lanjutkan transfer? (y/n): y
✓ Transfer sebesar Rp 50.000 berhasil dikirim.
Saldo tersisa : Rp 1.450.000
```

```
=== VIRTUAL ACCOUNT ===

Saldo tersedia : Rp 1.450.000
E-Wallet : Gopay
Nomor Virtual Account : 081234567890
Nominal (Rp) : 50000

Ringkasan Pembayaran VA
Layanan : Gopay
No. VA : 081234567890
Nominal : Rp 50.000
Sisa Saldo: Rp 1.400.000

Lanjutkan pembayaran VA? (y/n): y
✓ Pembayaran VA ke Gopay sebesar Rp 50.000 berhasil.
Saldo tersisa : Rp 1.400.000
```

Fitur Create terbagi menjadi tiga operasi utama: Transfer Bank, Virtual Account, dan Setor Tunai. Setiap operasi mengikuti alur yang sama: validasi input, konfirmasi ringkasan transaksi, dan penyimpanan ke file JSON.

Transfer Bank memungkinkan pengguna mengirim uang ke rekening bank lain. Sistem memvalidasi bahwa saldo mencukupi, nama pemilik rekening hanya berisi huruf, nomor rekening tepat 10 digit angka, dan nominal lebih dari nol. Setelah konfirmasi, saldo dikurangi dan transaksi baru ditambahkan ke list transaksi.

Virtual Account memungkinkan pembayaran ke e-wallet seperti Dana, GoPay, dan OVO. Nomor Virtual Account divalidasi minimal 8 digit angka.

Setor Tunai menambah saldo pengguna tanpa memerlukan nomor rekening. Transaksi ini otomatis diidentifikasi sebagai pemasukan oleh fungsi `is_pemasukan()` berdasarkan kata kunci pada field jenis dan bank.

```
def transfer_bank():
    data = baca_data()
    saldo = data["saldo"]
    nama = input_nama("Nama pemilik rekening : ")
    rek = input_rek()
    bank = input_nama("Nama bank : ")
    nominal = input_nominal()
    if nominal > saldo:
        print(" Saldo tidak mencukupi.")
    return
```

```
# ... konfirmasi dan simpan
```

3.3 Fitur Read (Mutasi Rekening)

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 1

--- Semua Transaksi ---
Halaman 1/1 | Total: 3 transaksi



| No | Tanggal             | Nama            | No. Rek/VA   |      | Nominal   | Bank | Jenis           |
|----|---------------------|-----------------|--------------|------|-----------|------|-----------------|
| 1  | 2026-06-26 09:21:07 | Setor Tunai ATM | -            | + Rp | 1.500.000 | Cash | Setor           |
| 2  | 2026-06-26 15:33:20 | Ironmen         | 1234567890   | - Rp | 50.000    | BCA  | Transfer Bank   |
| 3  | 2026-06-26 15:34:49 | Gopay           | 081234567890 | - Rp | 50.000    | VA   | Virtual Account |


```

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2

Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Top Up
4. Setor Tunai
Pilih Jenis (1-4): 1

--- Mutasi: Transfer Bank ---
Halaman 1/1 | Total: 1 transaksi



| No | Tanggal             | Nama    | No. Rek/VA |      | Nominal | Bank | Jenis         |
|----|---------------------|---------|------------|------|---------|------|---------------|
| 1  | 2026-06-26 15:33:20 | Ironmen | 1234567890 | - Rp | 50.000  | BCA  | Transfer Bank |


```

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2

Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Top Up
4. Setor Tunai
Pilih Jenis (1-4): 2

--- Mutasi: Virtual Account ---
Halaman 1/1 | Total: 1 transaksi



| No | Tanggal             | Nama  | No. Rek/VA   |      | Nominal | Bank | Jenis           |
|----|---------------------|-------|--------------|------|---------|------|-----------------|
| 1  | 2026-06-26 15:34:49 | Gopay | 081234567890 | - Rp | 50.000  | VA   | Virtual Account |


```

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2

Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Setor Tunai
Pilih Jenis (1-4): 3

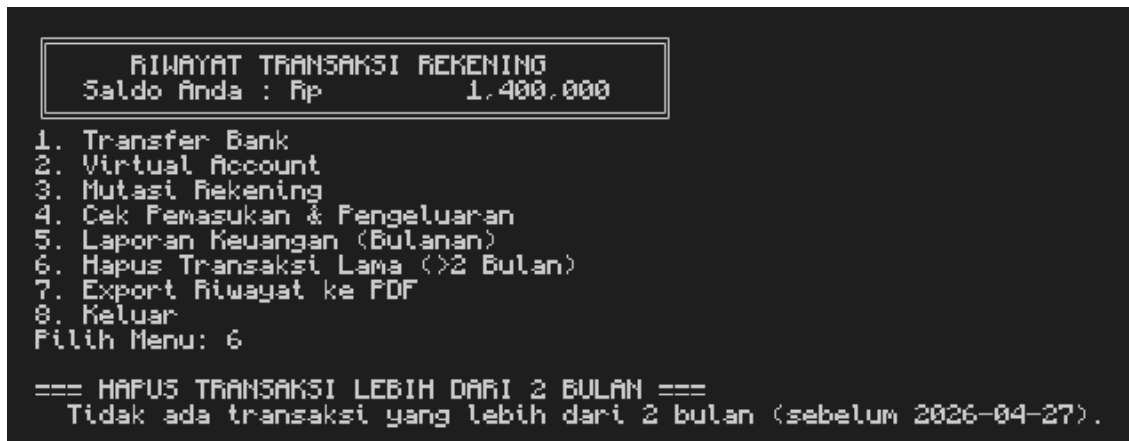
--- Mutasi: Setor Tunai ---
Halaman 1/1 | Total: 1 transaksi



| No | Tanggal             | Nama            | No. Rek/VA |      | Nominal   | Bank | Jenis |
|----|---------------------|-----------------|------------|------|-----------|------|-------|
| 1  | 2026-06-26 09:21:07 | Setor Tunai ATM | -          | + Rp | 1.500.000 | Cash | Setor |


```


3.4 Fitur Delete (Hapus Transaksi Lama)



Fitur Delete mengimplementasikan penghapusan transaksi yang berusia lebih dari 2 bulan (60 hari) dari tanggal sistem saat ini. Sistem terlebih dahulu menampilkan daftar transaksi yang akan dihapus beserta detailnya, kemudian meminta konfirmasi pengguna sebelum benar-benar menghapus.

Implementasi menggunakan datetime untuk menghitung selisih waktu dan memisahkan transaksi ke dalam dua list: `akan_dihapus` dan `akan_disimpan`. Hanya list `akan_disimpan` yang dituliskan kembali ke file JSON.

```
batas = datetime.now() - timedelta(days=60)
akan_dihapus = [t for t in transaksi if
datetime.strptime(t["tanggal"], "%Y-%m-%d %H:%M:%S") < batas]
akan_disimpan = [t for t in transaksi if
datetime.strptime(t["tanggal"], "%Y-%m-%d %H:%M:%S") >= batas]
```

3.5 Fitur Searching

Fitur searching diimplementasikan sebagai filter transaksi berdasarkan jenis menggunakan linear search. Pengguna memilih dari empat kategori: Transfer Bank, Virtual Account, Top Up, dan Setor Tunai. Sistem kemudian memfilter list transaksi dan menampilkan hasilnya dengan paginasi.

Untuk kategori Setor Tunai, sistem menggunakan fungsi `is_pemasukan()` yang melakukan keyword matching mencari kata kunci seperti "setor", "tunai", "cash" pada field jenis dan bank transaksi. Pendekatan ini lebih fleksibel dibandingkan pencocokan string tepat karena dapat menangani variasi data input.

```
def is_pemasukan(t: dict) -> bool:
```

```

teks_pencarian = (str(t.get("jenis", "")) + " " +
str(t.get("bank", ""))).lower()
kata_kunci = ["setor", "tunai", "cash", "pemasukan", "terima"]
return any(kata in teks_pencarian for kata in kata_kunci)

```

3.6 Fitur Sorting

Fitur sorting memungkinkan pengguna mengurutkan riwayat transaksi berdasarkan dua kriteria:

- Berdasarkan Tanggal (terbaru ke terlama): Menggunakan `sorted()` dengan key lambda yang membandingkan string tanggal. Format tanggal yang konsisten (YYYY-MM-DD HH:MM:SS) memungkinkan perbandingan string secara langsung tanpa perlu parsing ke objek datetime.
- Berdasarkan Nominal (terbesar ke terkecil): Menggunakan `sorted()` dengan key lambda yang mengkonversi nominal ke float sebelum dibandingkan. Konversi float diperlukan karena nilai nominal disimpan sebagai integer dalam JSON.

Kedua operasi sorting menggunakan parameter `reverse=True` untuk mendapatkan urutan menurun, dan menggunakan fungsi `sorted()` (bukan `.sort()`) agar list asli tidak dimodifikasi hasilnya hanya untuk tampilan, bukan perubahan permanen pada data.

3.7 Penyimpanan Data (File Handling)

Aplikasi menggunakan format JSON untuk persistensi data karena beberapa keunggulan: human-readable, didukung secara native oleh Python melalui modul `json`, struktur hirarkis yang sesuai dengan kebutuhan data, dan kompatibel dengan berbagai bahasa pemrograman jika diperlukan integrasi di masa depan.

Struktur file JSON:

```

{
  "saldo": 50000.0,
  "transaksi": [
    {
      "tanggal": "2026-06-03 13:12:31",
      "nama": "Setor Tunai ATM",
      "rek": "-",
      "nominal": 50000,
      "bank": "Cash",
      "jenis": "Setor"
    }
  ]
}

```

Cara membaca file: Fungsi `baca_data()` menggunakan `json.load()` untuk membaca dan mengurai file JSON menjadi dictionary Python. Fungsi ini juga menangani kasus file tidak ada (`FileNotFoundError`) dengan mengembalikan data default.

Cara menyimpan file: Fungsi `simpan_data()` menggunakan `json.dump()` dengan parameter `indent=2` untuk menghasilkan JSON yang terformat rapi, dan `ensure_ascii=False` untuk mendukung karakter non-ASCII (termasuk karakter Indonesia).

3.8 Validasi Input dan Error Handling

Aplikasi mengimplementasikan validasi input yang komprehensif melalui fungsi-fungsi validasi khusus:

Kesalahan Input	Penanganan
Input nama mengandung angka	Fungsi <code>input_nama()</code> memeriksa <code>any(c.isdigit() for c in nilai)</code> dan meminta input ulang jika ditemukan digit.
Nominal bukan angka atau negatif	Fungsi <code>input_nominal()</code> menggunakan <code>try/except ValueError</code> dan memeriksa nilai > 0 . Input diulang hingga valid.
Nomor rekening tidak 10 digit	Fungsi <code>input_rek()</code> memvalidasi <code>rek.isdigit()</code> and <code>len(rek) == 10</code> . Input diulang jika tidak sesuai.
Nomor VA kurang dari 8 digit	Fungsi <code>input_va()</code> memvalidasi <code>va.isdigit()</code> and <code>len(va) >= 8</code> .
Saldo tidak mencukupi untuk transfer	Nominal dibandingkan dengan saldo sebelum transaksi. Jika melebihi, transaksi dibatalkan dengan pesan error.
File JSON belum tersedia	Fungsi <code>baca_data()</code> mengembalikan data default <code>{"saldo": 0.0, "transaksi": []}</code> jika file belum ada.
Format tanggal tidak valid pada hapus transaksi	Blok <code>try/except ValueError</code> pada <code>hapus_transaksi_lama()</code> mempertahankan transaksi dengan format tanggal tidak valid alih-alih crash.
Nominal tidak valid pada sorting	Blok <code>try/except ValueError</code> pada <code>menu_tampilan()</code> untuk sorting nominal mengembalikan list asli tanpa pengurutan jika ada nominal yang tidak dapat dikonversi.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian dilakukan secara manual dengan menjalankan setiap fitur aplikasi dan memverifikasi output yang dihasilkan. Berikut adalah skenario pengujian yang dilakukan:

No	Skenario Pengujian	Hasil
1	Setor tunai dengan nominal valid (Rp 50.000)	BERHASIL. Saldo bertambah, transaksi tercatat di JSON dengan jenis "Setor" dan bank "Cash".
2	Setor tunai dengan nominal 0 atau negatif	BERHASIL. Program menampilkan pesan error dan meminta input ulang tanpa mengubah saldo.
3	Transfer bank dengan saldo mencukupi	BERHASIL. Saldo berkurang sesuai nominal, transaksi tercatat dengan detail nama, rekening, dan bank tujuan.
4	Transfer bank dengan saldo tidak mencukupi	BERHASIL. Program menampilkan pesan "Saldo tidak mencukupi" dan membatalkan transaksi tanpa perubahan data.
5	Transfer dengan nomor rekening kurang/lebih dari 10 digit	BERHASIL. Program menolak input dan meminta nomor rekening ulang.
6	Transfer dengan nama yang mengandung angka	BERHASIL. Program menolak input dan meminta nama ulang.
7	Pembayaran Virtual Account ke Dana	BERHASIL. Saldo berkurang, transaksi tercatat dengan jenis "Virtual Account" dan bank sesuai provider.
8	Tampilkan semua riwayat transaksi	BERHASIL. Seluruh transaksi ditampilkan dengan format tabel, tanda +/- pada nominal, dan paginasi 10 per halaman.
9	Filter transaksi berdasarkan jenis "Transfer Bank"	BERHASIL. Hanya menampilkan transaksi dengan jenis Transfer Bank.
10	Sort transaksi berdasarkan tanggal (terbaru ke terlama)	BERHASIL. Transaksi terurut dari tanggal terbaru ke terlama.
11	Sort transaksi berdasarkan nominal (terbesar ke terkecil)	BERHASIL. Transaksi terurut dari nominal terbesar ke terkecil.

12	Cek laporan keuangan per bulan	BERHASIL. Menampilkan ringkasan pemasukan dan pengeluaran per bulan.
13	Laporan per bank/layanan	BERHASIL. Menampilkan subtotal pengeluaran per bank beserta rincian per jenis transaksi.
14	Hapus transaksi lebih dari 2 bulan	BERHASIL. Menampilkan daftar transaksi yang akan dihapus, meminta konfirmasi, lalu menghapus setelah dikonfirmasi.
15	Export riwayat ke PDF	BERHASIL. File PDF berhasil digenerate dengan nama menyertakan timestamp.
16	Menjalankan program saat file JSON belum ada	BERHASIL. Program membuat data default dengan saldo 0 dan list transaksi kosong.
17	Membatalkan transaksi saat konfirmasi (pilih "n")	BERHASIL. Transaksi dibatalkan, data tidak berubah.

4.2 Kelebihan Sistem

- Data tersimpan permanen menggunakan format JSON, sehingga riwayat transaksi tidak hilang meski program ditutup dan dibuka kembali.
- Validasi input yang komprehensif mencegah data tidak valid masuk ke sistem, menjaga integritas data transaksi.
- Sistem tidak mudah crash seluruh titik potensial error telah dilindungi oleh blok try/except dengan pesan error yang informatif.
- Menu yang hierarkis dan intuitif memudahkan navigasi meski berbasis teks; pengguna selalu diberikan pilihan untuk kembali ke menu sebelumnya.
- Paginasi pada tampilan riwayat transaksi memastikan program tetap nyaman digunakan meski jumlah transaksi sangat banyak.
- Deteksi otomatis jenis transaksi (pemasukan vs pengeluaran) menggunakan keyword matching yang fleksibel.
- Fitur export PDF memungkinkan pengguna menyimpan laporan dalam format yang mudah dibagikan dan dicetak.
- Format JSON yang human-readable memudahkan debugging dan verifikasi data secara langsung.

4.3 Keterbatasan Sistem

- Masih berbasis CLI sehingga tidak memiliki antarmuka grafis yang dapat lebih mudah digunakan oleh pengguna awam.
- Belum mendukung database relasional; penggunaan file JSON kurang optimal untuk dataset yang sangat besar karena seluruh file dibaca dan ditulis ulang setiap operasi.
- Belum mendukung multiuser aplikasi hanya dapat digunakan oleh satu pengguna dengan satu akun tanpa sistem autentikasi.
- Fitur update (edit) transaksi belum tersedia; pengguna tidak dapat memperbaiki transaksi yang sudah tersimpan selain dengan menghapus seluruh transaksi lama.
- Export PDF menggunakan layout yang sederhana tanpa grafik atau visualisasi data.
- Belum ada fitur pencarian berdasarkan nama atau nominal tertentu (hanya filter berdasarkan jenis transaksi).

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Data transaksi hilang saat program ditutup pada versi awal pengembangan yang menggunakan variabel global.	Mengimplementasikan file handling menggunakan format JSON. Setiap perubahan data langsung ditulis ke file menggunakan fungsi <code>simpan_data()</code> , dan data dibaca ulang dari file di awal setiap operasi menggunakan <code>baca_data()</code> .
Program crash saat pengguna memasukkan huruf pada field nominal atau nomor rekening.	Membungkus konversi tipe data dalam blok <code>try/except ValueError</code> dan mengulang permintaan input hingga format yang valid diberikan. Implementasi dalam fungsi <code>input_nominal()</code> , <code>input_rek()</code> , dan <code>input_va()</code> .
Kesulitan membedakan transaksi pemasukan dan pengeluaran secara programatik karena tidak ada field khusus untuk arah arus kas.	Mengimplementasikan fungsi <code>is_pemasukan()</code> yang menggunakan keyword matching pada gabungan field jenis dan bank. Pendekatan ini fleksibel dan tidak memerlukan perubahan struktur data yang sudah ada.
Tampilan daftar transaksi yang panjang sulit dibaca dan membuat terminal tidak nyaman digunakan.	Mengimplementasikan sistem paginasi melalui fungsi <code>cetak_pagination()</code> yang menampilkan 10 transaksi per halaman dengan navigasi N (next), P (previous), dan Q (quit).
Penentuan format penyimpanan data awal (CSV vs JSON) yang diperdebatkan dalam tim.	Memilih JSON karena lebih sesuai dengan struktur data hierarkis (saldo + list transaksi), didukung natively oleh Python, dan human-readable untuk debugging. CSV lebih cocok untuk data tabular flat tanpa nesting.
Sorting berdasarkan nominal gagal jika ada data nominal yang corrupt (tidak bisa dikonversi ke float).	Menambahkan blok <code>try/except ValueError</code> pada operasi sorting nominal di <code>menu_tampilan()</code> . Jika sorting gagal, sistem menampilkan data dalam urutan asli tanpa crash.
Instalasi pustaka ReportLab yang tidak tersedia secara default memerlukan langkah tambahan.	Mendokumentasikan kebutuhan instalasi dengan perintah <code>pip install reportlab</code> di README. Untuk environment yang tidak dapat menginstal pustaka eksternal, fitur export PDF dapat dilewati tanpa memengaruhi fitur lainnya.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Syahirah Yusriyah Putri	1. Menu Utama 2. Reportlab (Ekspot PDF)
Muhamad Ilham Nurhakim	1. ATM Mini (Flask) 2. Validasi Pemasukkan 3. Rincian Transaksi
Ahmad Maulana Abimanyu	1. Validasi Transaksi 2. Transfer 3. Laporan Keuangan

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan:

- Proyek DUID Banking CLI berhasil diimplementasikan sebagai aplikasi manajemen keuangan berbasis terminal yang fungsional, mampu menjalankan operasi perbankan dasar meliputi setor tunai, transfer bank, pembayaran virtual account, penampilan riwayat transaksi, laporan keuangan, dan export PDF.
- Penerapan struktur data Dictionary, List, dan defaultdict terbukti efektif dalam menyimpan dan memproses data transaksi. File JSON sebagai media persistensi berhasil menjamin kontinuitas data antar sesi program, membuktikan bahwa file handling yang tepat dapat menggantikan kebutuhan database untuk aplikasi skala kecil.
- Implementasi algoritma Linear Search untuk filtering dan Timsort (melalui built-in sorted()) untuk pengurutan berjalan dengan baik sesuai kebutuhan. Validasi input berlapis terbukti meningkatkan robustness aplikasi secara signifikan, menghilangkan kemungkinan crash akibat input tidak valid.

Saran Pengembangan

Untuk pengembangan aplikasi di masa mendatang, disarankan:

- Implementasi antarmuka grafis (GUI) menggunakan framework seperti Tkinter atau PyQt untuk meningkatkan kemudahan penggunaan oleh pengguna awam yang tidak familiar dengan CLI.
- Migrasi penyimpanan data dari file JSON ke database relasional seperti SQLite untuk performa yang lebih baik pada dataset besar, serta kemampuan query yang lebih powerful.
- Penambahan sistem autentikasi multi-user dengan enkripsi data untuk mendukung penggunaan oleh lebih dari satu pengguna pada satu perangkat.
- Penambahan fitur pencarian transaksi berdasarkan nama, nominal tertentu, atau rentang tanggal untuk memudahkan pengguna menemukan transaksi spesifik.
- Implementasi fitur notifikasi atau reminder untuk transaksi rutin atau batas saldo minimum.

- Pengembangan fitur visualisasi data berupa grafik tren pengeluaran bulanan yang dapat diintegrasikan ke dalam laporan PDF.
- Penambahan fitur backup dan restore data untuk melindungi data dari kehilangan akibat kerusakan file.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python. Hoboken, NJ: Wiley, 2013.
- [3] B. Miller and D. Ranum, Problem Solving with Algorithms and Data Structures Using Python. Sherwood, OR: Franklin, Beedle & Associates, 2011.
- [4] Python Software Foundation, "Python 3 Documentation json JSON encoder and decoder," 2024. [Online]. Available: <https://docs.python.org/3/library/json.html>
- [5] Python Software Foundation, "Python 3 Documentation collections Container datatypes," 2024. [Online]. Available: <https://docs.python.org/3/library/collections.html>
- [6] Python Software Foundation, "Python 3 Documentation datetime Basic date and time types," 2024. [Online]. Available: <https://docs.python.org/3/library/datetime.html>
- [7] ReportLab Group, "ReportLab User Guide," 2023. [Online]. Available: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [8] L. Ramalho, Fluent Python: Clear, Concise, and Effective Programming, 2nd ed. Sebastopol, CA: O'Reilly Media, 2022.

LAMPIRAN

Lampiran A. Link GitHub

https://github.com/vrvatos/ASD_A_19

Lampiran B. Screenshot Program

```
RIWAYAT TRANSAKSI REKENING
Saldo Anda : Rp      1.500.000

1. Transfer Bank
2. Virtual Account
3. Mutasi Rekening
4. Cek Pemasukan & Pengeluaran
5. Laporan Keuangan (Bulanan)
6. Hapus Transaksi Lama (>2 Bulan)
7. Export Riwayat ke PDF
8. Keluar
Pilih Menu:
```

```
=== VIRTUAL ACCOUNT ===

Saldo tersedia : Rp 1.450.000
E-Wallet : Gopay
Nomor Virtual Account : 081234567890
Nominal (Rp) : 50000

Ringkasan Pembayaran VA
Layanan : Gopay
No. VA : 081234567890
Nominal : Rp 50.000
Sisa Saldo: Rp 1.400.000

Lanjutkan pembayaran VA? (y/n): y

✓ Pembayaran VA ke Gopay sebesar Rp 50.000 berhasil.
Saldo tersisa : Rp 1.400.000
```

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 1

--- Semua Transaksi ---
Halaman 1/1 | Total: 3 transaksi



| No | Tanggal             | Nama            | No. Rek/VA   |      | Nominal   | Bank | Jenis           |
|----|---------------------|-----------------|--------------|------|-----------|------|-----------------|
| 1  | 2026-06-26 09:21:07 | Setor Tunai ATM | -            | + Rp | 1.500.000 | Cash | Setor           |
| 2  | 2026-06-26 15:33:28 | Ironmen         | 1234567890   | - Rp | 50.000    | BCA  | Transfer Bank   |
| 3  | 2026-06-26 15:34:49 | Gopay           | 081234567890 | - Rp | 50.000    | VA   | Virtual Account |


```

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2
```

```
Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Top Up
4. Setor Tunai
Pilih Jenis (1-4): 1
```

```
— Mutasi: Transfer Bank —
Halaman 1/1 | Total: 1 transaksi
```

No	Tanggal	Nama	No. Rek/VA		Nominal	Bank	Jenis
1	2026-06-26 15:33:20	Ironmen	1234567890	- Rp	50.000	BCA	Transfer Bank

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2
```

```
Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Top Up
4. Setor Tunai
Pilih Jenis (1-4): 2
```

```
— Mutasi: Virtual Account —
Halaman 1/1 | Total: 1 transaksi
```

No	Tanggal	Nama	No. Rek/VA		Nominal	Bank	Jenis
1	2026-06-26 15:34:49	Gopay	081234567890	- Rp	50.000	VA	Virtual Account

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 2
```

```
Pilih Filter:
1. Transfer Bank
2. Virtual Account
3. Setor Tunai
Pilih Jenis (1-4): 3
```

```
— Mutasi: Setor Tunai —
Halaman 1/1 | Total: 1 transaksi
```

No	Tanggal	Nama	No. Rek/VA		Nominal	Bank	Jenis
1	2026-06-26 09:21:07	Setor Tunai ATM	-	+ Rp	1.500.000	Cash	Setor

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 3
```

```
— Urutan Berdasarkan Tanggal (Terbaru → Terlama) —
Halaman 1/1 | Total: 3 transaksi
```

No	Tanggal	Nama	No. Rek/VA		Nominal	Bank	Jenis
1	2026-06-26 15:34:49	Gopay	081234567890	- Rp	50.000	VA	Virtual Account
2	2026-06-26 15:33:20	Ironmen	1234567890	- Rp	50.000	BCA	Transfer Bank
3	2026-06-26 09:21:07	Setor Tunai ATM	-	+ Rp	1.500.000	Cash	Setor

```
=== Mutasi Rekening ===
1. Tampilkan Semua Riwayat
2. Tampilkan Berdasarkan Jenis Transaksi
3. Tampilkan Berdasarkan Tanggal
4. Tampilkan Berdasarkan Nominal
5. Kembali ke Menu Utama
Pilih opsi (1-5): 4
```

```
— Urutan Berdasarkan Nominal (Terbesar → Terkecil) —
Halaman 1/1 | Total: 3 transaksi
```

No	Tanggal	Nama	No. Rek/VA		Nominal	Bank	Jenis
1	2026-06-26 09:21:07	Setor Tunai ATM	-	+ Rp	1.500.000	Cash	Setor
2	2026-06-26 15:33:20	Ironmen	1234567890	- Rp	50.000	BCA	Transfer Bank
3	2026-06-26 15:34:49	Gopay	081234567890	- Rp	50.000	VA	Virtual Account

RIWAYAT TRANSAKSI REKENING
Saldo Anda : Rp 1.400.000

1. Transfer Bank
 2. Virtual Account
 3. Mutasi Rekening
 4. Cek Pemasukan & Pengeluaran
 5. Laporan Keuangan (Bulanan)
 6. Hapus Transaksi Lama (>2 Bulan)
 7. Export Riwayat ke PDF
 8. Keluar
- Pilih Menu: 4

=== CEK PEMASUKAN & PENGELUARAN ===

[TOTAL PEMASUKAN (+)]
Setor Tunai : Rp 1.500.000

[TOTAL PENGELUARAN (-)]
Transfer Bank : Rp 50.000
Virtual Account : Rp 50.000

TOTAL PENGELUARAN : Rp 100.000

RIWAYAT TRANSAKSI REKENING
Saldo Anda : Rp 1.400.000

1. Transfer Bank
 2. Virtual Account
 3. Mutasi Rekening
 4. Cek Pemasukan & Pengeluaran
 5. Laporan Keuangan (Bulanan)
 6. Hapus Transaksi Lama (>2 Bulan)
 7. Export Riwayat ke PDF
 8. Keluar
- Pilih Menu: 5

=== LAPORAN KEUANGAN ===

1. Ringkasan per Bulan (Pemasukan dan Pengeluaran)
2. Ringkasan per Bank/Layanan (hanya Pengeluaran)
3. Kembali

Pilih opsi (1-3): 1

Bulan		Pemasukan (+)		Pengeluaran (-)
2026-06	Rp	1.500.000	Rp	100.000

=== LAPORAN KEUANGAN ===

1. Ringkasan per Bulan (Pemasukan dan Pengeluaran)
2. Ringkasan per Bank/Layanan (hanya Pengeluaran)
3. Kembali

Pilih opsi (1-3): 2

[BCA]

Transfer Bank	: Rp	50.000
Subtotal	: Rp	50.000

[VA]

Virtual Account	: Rp	50.000
Subtotal	: Rp	50.000

=== LAPORAN KEUANGAN ===

1. Ringkasan per Bulan (Pemasukan dan Pengeluaran)
2. Ringkasan per Bank/Layanan (hanya Pengeluaran)
3. Kembali

Pilih opsi (1-3):

RIWAYAT TRANSAKSI REKENING

Saldo Anda : Rp 1.400.000

1. Transfer Bank
 2. Virtual Account
 3. Mutasi Rekening
 4. Cek Pemasukan & Pengeluaran
 5. Laporan Keuangan (Bulanan)
 6. Hapus Transaksi Lama (>2 Bulan)
 7. Export Riwayat ke PDF
 8. Keluar
- Pilih Menu: 6

=== HAPUS TRANSAKSI LEBIH DARI 2 BULAN ===

Tidak ada transaksi yang lebih dari 2 bulan (sebelum 2026-04-27).

RIWAYAT TRANSAKSI REKENING

Saldo Anda : Rp 1.400.000

1. Transfer Bank
 2. Virtual Account
 3. Mutasi Rekening
 4. Cek Pemasukan & Pengeluaran
 5. Laporan Keuangan (Bulanan)
 6. Hapus Transaksi Lama (>2 Bulan)
 7. Export Riwayat ke PDF
 8. Keluar
- Pilih Menu: 7

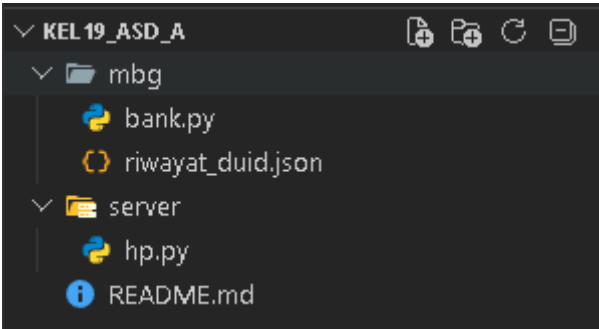
✓ PDF berhasil dibuat:

laporan_transaksi_20260626_161250.pdf

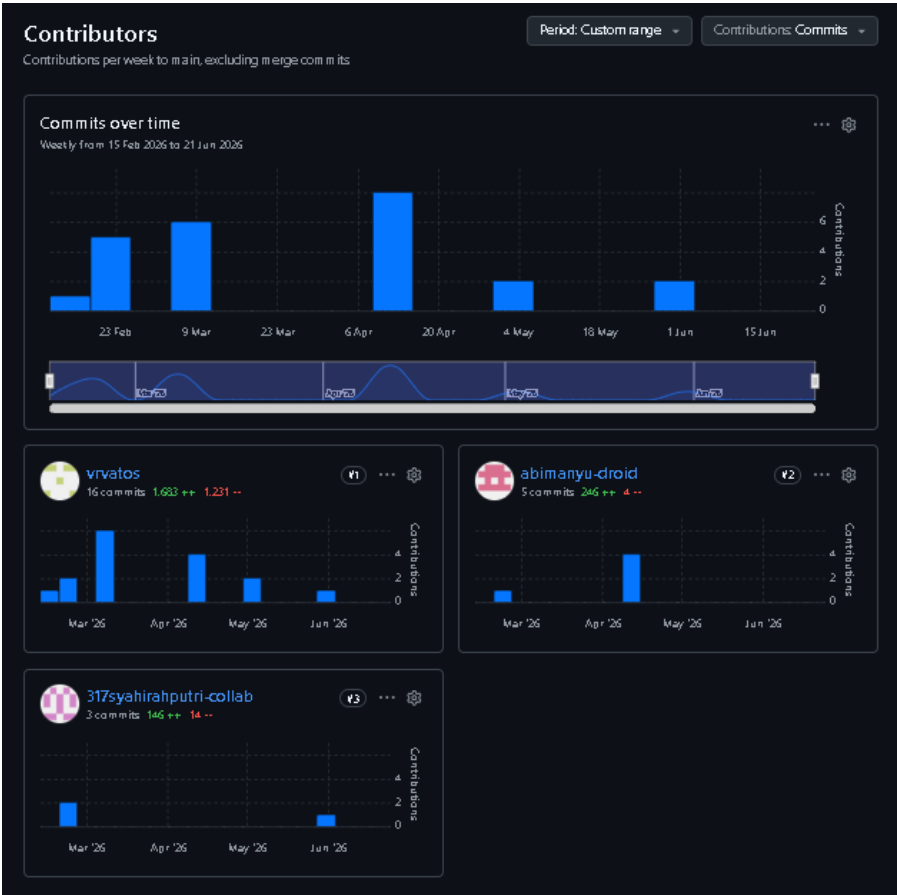
LAPORAN RIWAYAT TRANSAKSI

Tanggal	Nama	Rek/VA	Nominal	Bank	Jenis
2026-06-26 09:21:07	Setor Tunai ATM	-	Rp 1,500,000	Cash	Setor
2026-06-26 15:33:20	Ironmen	1234567890	Rp 50,000	BCA	Transfer Bank
2026-06-26 15:34:49	Gopay	081234567890	Rp 50,000	VA	Virtual Account

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History GitHub



Lampiran E. Source Code Utama

Berikut adalah source code lengkap aplikasi DUID Banking CLI (bank.py):

```
import os
import json
from datetime import datetime, timedelta
from collections import defaultdict
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle,
Paragraph, Spacer
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet

# Nama file JSON untuk menyimpan seluruh data aplikasi
nama_file = "riwayat_duid.json"

# Jumlah transaksi yang ditampilkan per halaman
page = 10

# Daftar jenis transaksi untuk filter mutasi
validitas = {
    "1": "Transfer Bank",
    "2": "Virtual Account",
    "3": "Setor Tunai"
}

#
=====
# validasi pemasukkan
#
=====

def is_pemasukan(t: dict) -> bool:
    """
    Fungsi untuk mendeteksi apakah transaksi adalah Pemasukan.
    Jika ada kata berikut, maka dianggap Pemasukan.
    """
    teks_pencarian = (str(t.get('jenis', '')) + " " +
str(t.get('bank', ''))).lower()
    kata_kunci = ['setor', 'tunai', 'cash', 'pemasukan', 'terima']

    return any(kata in teks_pencarian for kata in kata_kunci)

#
=====
# read n write
#
=====
```

```

def baca_data() -> dict:
    default = {"saldo": 0.0, "transaksi": []}
    if not os.path.exists(nama_file):
        return default
    with open(nama_file, "r") as f:
        data = json.load(f)
        if "saldo" not in data:
            data["saldo"] = 0.0
        if "transaksi" not in data:
            data["transaksi"] = []
        return data

def simpan_data(data: dict):
    with open(nama_file, "w") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)

def baca_saldo() -> float:
    return baca_data()["saldo"]

def baca_transaksi() -> list:
    return baca_data()["transaksi"]

#
=====
# Rincian Transaksi
#
=====

def cetak_header():
    print(f"\n{'No':<4} {'Tanggal':<20} {'Nama':<18} {'No. Rek/VA':<15} "
          f"{'Nominal':>18} {'Bank':<10} {'Jenis'}")
    print("-" * 105)

def cetak_baris(i, t):
    try:
        nominal_val = int(float(t['nominal']))
        if is_pemasukan(t):
            nominal_fmt = f"+ Rp {nominal_val:>11,}" # Tanda + untuk
uang masuk
        else:
            nominal_fmt = f"- Rp {nominal_val:>11,}" # Tanda - untuk
uang keluar
    except (ValueError, KeyError):
        nominal_fmt = f"{'??:>18}"

    print(f"{'i':<4} {t['tanggal']:<20} {t['nama']:<18} {t['rek']:<15} "
          f"{nominal_fmt} {t['bank']:<10} {t['jenis']}")

def cetak_pagination(data, judul=""):

```

```

if not data:
    print("  (Tidak ada data.)")
    return

total      = len(data)
total_hal = (total + page - 1) // page
halaman    = 1

while True:
    mulai    = (halaman - 1) * page
    akhir    = mulai + page
    potongan = data[mulai:akhir]

    if judul:
        print(f"\n—— {judul} ——")

    print(f"  Halaman {halaman}/{total_hal}  |  Total: {total}
transaksi")
    cetak_header()

    for i, t in enumerate(potongan, mulai + 1):
        cetak_baris(i, t)

    if total_hal == 1:
        break

    print(f"\n  [N] Halaman Berikutnya  [P] Halaman
Sebelumnya  [Q] Kembali")
    nav = input("  Navigasi: ").strip().upper()

    if nav == "N":
        if halaman < total_hal:
            halaman += 1
        else:
            print("  Sudah di halaman terakhir.")
    elif nav == "P":
        if halaman > 1:
            halaman -= 1
        else:
            print("  Sudah di halaman pertama.")
    elif nav == "Q":
        break
    else:
        print("  Pilihan tidak valid.")
        break

```

#

validasi transaksi

#

```

def input_nama(label):
    while True:
        nilai = input(label).strip()
        if not nilai:
            print(" Tidak boleh kosong.")
        elif any(c.isdigit() for c in nilai):
            print(" Nama tidak boleh mengandung angka.")
        else:
            return nilai

def input_nominal():
    while True:
        raw = input("Nominal (Rp) : ").strip().replace(",",
""").replace(".", "")
        try:
            nilai = float(raw)
            if nilai <= 0:
                print(" Nominal harus lebih dari 0.")
            else:
                return nilai
        except ValueError:
            print(" Input tidak valid, masukkan angka saja.")

def input_rek():
    while True:
        rek = input("Nomor rekening : ").strip()
        if rek.isdigit() and len(rek) == 10:
            return rek
        print(" Nomor rekening harus tepat 10 digit angka.")

def input_va():
    while True:
        va = input("Nomor Virtual Account : ").strip()
        if va.isdigit() and len(va) >= 8:
            return va
        print(" Nomor VA tidak valid (harus angka, minimal 8
digit).")

def konfirmasi(pesan="Simpan transaksi ini? (y/n): "):
    while True:
        jwb = input(pesan).strip().lower()
        if jwb in ("y", "n"):
            return jwb == "y"
        print(" Masukkan 'y' untuk Ya atau 'n' untuk Tidak.")

#
=====
# transfer
#
=====
=====

```

```

def transfer_bank():
    print("\n=== TRANSFER BANK ===")
    data = baca_data()
    saldo = data["saldo"]

    print(f"\n Saldo tersedia : Rp {saldo:,.0f}")
    if saldo <= 0:
        print(" Saldo Anda tidak mencukupi.")
        return

    jenis = "Transfer Bank"
    nama = input_nama("Nama pemilik rekening : ")
    rek = input_rek()
    bank = input_nama("Nama bank : ")
    nominal = input_nominal()

    if nominal > saldo:
        print(f"\n X Saldo tidak mencukupi. Maksimum: Rp {saldo:,.0f}")
        return

    print("\n┌ Ringkasan Transaksi ───────────────────┐")
    print(f"| Jenis : {jenis}|")
    print(f"| Nama : {nama}|")
    print(f"| Rek : {rek}|")
    print(f"| Bank : {bank}|")
    print(f"| Nominal : Rp {int(nominal):,}|")
    print(f"| Sisa Saldo: Rp {saldo - nominal:,.0f}|")
    print("└──────────────────────────────────────────┘")

    if not konfirmasi("Lanjutkan transfer? (y/n): "):
        print(" Transaksi dibatalkan.")
        return

    transaksi_baru = {
        "tanggal": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "nama": nama,
        "rek": rek,
        "nominal": int(nominal),
        "bank": bank,
        "jenis": jenis
    }

    data["transaksi"].append(transaksi_baru)
    data["saldo"] = saldo - nominal
    simpan_data(data)

    print(f"\n✓ Transfer sebesar Rp {int(nominal):,} berhasil dikirim.")
    print(f" Saldo tersisa : Rp {data['saldo']:,.0f}")

def virtual_account():

```

```

print("\n=== VIRTUAL ACCOUNT ===")
data = baca_data()
saldo = data["saldo"]

print(f"\n Saldo tersedia : Rp {saldo:,.0f}")
if saldo <= 0:
    print(" Saldo Anda tidak mencukupi.")
    return

jenis = "Virtual Account"
provider = input_nama("E-Wallet : ")
no_va = input_va()
nominal = input_nominal()

if nominal > saldo:
    print(f"\n X Saldo tidak mencukupi. Maksimum: Rp {saldo:,.0f}")
    return

print("\n┌ Ringkasan Pembayaran VA ───────────┐")
print(f"│ Layanan : {provider}│")
print(f"│ No. VA : {no_va}│")
print(f"│ Nominal : Rp {int(nominal):,}│")
print(f"│ Sisa Saldo: Rp {saldo - nominal:,.0f}│")
print("└──────────────────────────────────┘")

if not konfirmasi("Lanjutkan pembayaran VA? (y/n): "):
    print(" Pembayaran dibatalkan.")
    return

transaksi_baru = {
    "tanggal": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    "nama" : provider,
    "rek" : no_va,
    "nominal": int(nominal),
    "bank" : "VA",
    "jenis" : jenis
}

data["transaksi"].append(transaksi_baru)
data["saldo"] = saldo - nominal
simpan_data(data)

print(f"\n✓ Pembayaran VA ke {provider} sebesar Rp {int(nominal):,} berhasil.")
print(f" Saldo tersisa : Rp {data['saldo']:,.0f}")

#
=====
# riwayat

```

```

#
=====

def cek_rekap_keuangan():
    print("\n=== CEK PEMASUKAN & PENGELUARAN ===")
    transaksi = baca_transaksi()

    if not transaksi:
        print("Belum ada transaksi.")
        return

    total_masuk = 0.0
    per_jenis_keluar = defaultdict(float)

    for t in transaksi:
        if is_pemasukan(t):
            total_masuk += float(t['nominal'])
        else:
            per_jenis_keluar[t['jenis']] += float(t['nominal'])

    total_keluar = sum(per_jenis_keluar.values())

    print("\n [ TOTAL PEMASUKAN (+)]")
    print(f" Setor Tunai : Rp {total_masuk:>15,.0f}")

    print("\n [ TOTAL PENGELUARAN (-)]")
    if not per_jenis_keluar:
        print(" (Belum ada pengeluaran)")
    else:
        for jenis, jumlah in per_jenis_keluar.items():
            print(f" {jenis:<20} : Rp {jumlah:>15,.0f}")

    print(f" {'-' * 42}")
    print(f" {'TOTAL PENGELUARAN':<20} : Rp {total_keluar:>15,.0f}")

def menu_laporan():
    while True:
        print("\n=== LAPORAN KEUANGAN ===")
        print("1. Ringkasan per Bulan (Pemasukan dan Pengeluaran)")
        print("2. Ringkasan per Bank/Layanan (hanya Pengeluaran)")
        print("3. Kembali")
        pilih = input("Pilih opsi (1-3): ").strip()
        transaksi = baca_transaksi()

        if pilih == '1':
            if not transaksi:
                print("Belum ada transaksi.")
                continue

            per_bulan = defaultdict(lambda: {"masuk": 0.0, "keluar":
0.0})

```

```

        for t in transaksi:
            bulan = t['tanggal'][:7]
            if is_pemasukan(t):
                per_bulan[bulan]["masuk"] += float(t['nominal'])
            else:
                per_bulan[bulan]["keluar"] += float(t['nominal'])

        print(f"\n  {'Bulan':<10} | {'Pemasukan (+)':>18} |
{'Pengeluaran (-)':>18}")
        print("  " + "-" * 54)
        for bulan in sorted(per_bulan):
            masuk = per_bulan[bulan]["masuk"]
            keluar = per_bulan[bulan]["keluar"]
            print(f"    {bulan:<10} | Rp {masuk:>15,.0f} | Rp
{keluar:>15,.0f}")

    elif pilih == '2':
        if not transaksi:
            print("Belum ada transaksi.")
            continue

        per_bank = defaultdict(lambda: defaultdict(float))
        for t in transaksi:
            if not is_pemasukan(t):
                per_bank[t['bank']][t['jenis']] +=
float(t['nominal'])

        print()
        for bank in sorted(per_bank):
            subtotal = sum(per_bank[bank].values())
            print(f"    [ {bank} ]")
            for jenis, jumlah in per_bank[bank].items():
                print(f"        {jenis:<15}: Rp {jumlah:>13,.0f}")
            print(f"        {'Subtotal':<15}: Rp {subtotal:>13,.0f}")
            print()

    elif pilih == '3':
        break
    else:
        print("Opsi tidak valid.")

def menu_tampilan():
    while True:
        print("\n=== Mutasi Rekening ===")
        print("1. Tampilkan Semua Riwayat")
        print("2. Tampilkan Berdasarkan Jenis Transaksi")
        print("3. Tampilkan Berdasarkan Tanggal")
        print("4. Tampilkan Berdasarkan Nominal")
        print("5. Kembali ke Menu Utama")
        pilih = input("Pilih opsi (1-5): ").strip()
        transaksi = baca_transaksi()

        if pilih == '1':

```

```

        cetak_pagination(transaksi, "Semua Transaksi")

    elif pilih == '2':
        print("\n Pilih Filter:")
        for k, v in validitas.items():
            print(f" {k}. {v}")
        pilih_jenis = input("Pilih jenis (1-4): ").strip()

        if pilih_jenis not in validitas:
            print(" Pilihan tidak valid.")
            continue

        jenis = validitas[pilih_jenis]
        hasil = [t for t in transaksi if t['jenis'] == jenis or
(jenis == "Setor Tunai" and is_pemasukan(t))]
        cetak_pagination(hasil, f"Mutasi: {jenis}")

    elif pilih == '3':
        urutan = sorted(transaksi, key=lambda x: x['tanggal'],
reverse=True)
        cetak_pagination(urutan, "Urutan Berdasarkan Tanggal
(Terbaru → Terlama)")

    elif pilih == '4':
        try:
            urutan = sorted(transaksi, key=lambda x:
float(x['nominal']), reverse=True)
        except ValueError:
            urutan = transaksi
        cetak_pagination(urutan, "Urutan Berdasarkan Nominal
(Terbesar → Terkecil)")

    elif pilih == '5':
        break
    else:
        print("Opsi tidak valid. Silakan pilih antara 1-5.")

def hapus_transaksi_lama():
    print("\n=== HAPUS TRANSAKSI LEBIH DARI 2 BULAN ===")
    data = baca_data()
    transaksi = data["transaksi"]

    if not transaksi:
        print("Belum ada transaksi.")
        return

    batas = datetime.now() - timedelta(days=60)
    akan_dihapus = []
    akan_disimpan = []

    for t in transaksi:
        try:
            tgl = datetime.strptime(t['tanggal'], "%Y-%m-%d %H:%M:%S")

```

```

        if tgl < batas:
            akan_dihapus.append(t)
        else:
            akan_disimpan.append(t)
    except ValueError:
        akan_disimpan.append(t)

    if not akan_dihapus:
        print(f"    Tidak ada transaksi yang lebih dari 2 bulan (sebelum
{batas.strftime('%Y-%m-%d')}).")
        return

    print(f"\n    Ditemukan {len(akan_dihapus)} transaksi sebelum
{batas.strftime('%Y-%m-%d')} yang akan dihapus:\n")
    cetak_header()
    for i, t in enumerate(akan_dihapus, 1):
        cetak_baris(i, t)

    if not konfirmasi(f"Hapus {len(akan_dihapus)} transaksi lama ini?
(y/n): "):
        print("    Penghapusan dibatalkan.")
        return

    data["transaksi"] = akan_disimpan
    simpan_data(data)
    print(f"\n✓ {len(akan_dihapus)} transaksi lama berhasil dihapus.")
    print(f"    {len(akan_disimpan)} transaksi tersisa dalam riwayat.")

#
=====
=====
# menu utama
#
=====
=====
def export_pdf():
    transaksi = baca_transaksi()

    if not transaksi:
        print("Belum ada transaksi untuk diexport.")
        return

    nama_pdf =
f"laporan_transaksi_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"

    doc = SimpleDocTemplate(nama_pdf)
    styles = getSampleStyleSheet()

    elemen = []

    judul = Paragraph("LAPORAN RIWAYAT TRANSAKSI", styles['Title'])
    elemen.append(judul)
    elemen.append(Spacer(1, 12))

```

```

data_tabel = [[
    "Tanggal",
    "Nama",
    "Rek/VA",
    "Nominal",
    "Bank",
    "Jenis"
]]

for t in transaksi:
    data_tabel.append([
        t["tanggal"],
        t["nama"],
        t["rek"],
        f"Rp {int(t['nominal']):,}",
        t["bank"],
        t["jenis"]
    ])

tabel = Table(data_tabel)

tabel.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,0), colors.grey),
    ('TEXTCOLOR', (0,0), (-1,0), colors.whitesmoke),

    ('GRID', (0,0), (-1,-1), 1, colors.black),

    ('FONTNAME', (0,0), (-1,0), 'Helvetica-Bold'),

    ('BACKGROUND', (0,1), (-1,-1), colors.beige)
]))

elemen.append(tabel)

doc.build(elemen)

print(f"\n✓ PDF berhasil dibuat:")
print(f"    {nama_pdf}")

def main():
    while True:
        saldo = baca_saldo()

        print("\n" + "┌" + "RIWAYAT TRANSAKSI REKENING" + "┐")
        print(f"└ Saldo Anda : Rp {saldo:>17,.0f} ┘")
        print("1. Transfer Bank")
        print("2. Virtual Account")
        print("3. Mutasi Rekening")
        print("4. Cek Pemasukan & Pengeluaran")
        print("5. Laporan Keuangan (Bulanan)")

```

```
print("6. Hapus Transaksi Lama (>2 Bulan)")
print("7. Export Riwayat ke PDF")
print("8. Keluar")

pilihan = input("Pilih Menu: ").strip()

if pilihan == '1':
    transfer_bank()
elif pilihan == '2':
    virtual_account()
elif pilihan == '3':
    menu_tampilan()
elif pilihan == '4':
    cek_rekap_keuangan()
elif pilihan == '5':
    menu_laporan()
elif pilihan == '6':
    hapus_transaksi_lama()
elif pilihan == '7':
    export_pdf()
elif pilihan == '8':
    print("Sistem Berhenti. Data Anda aman di
'riwayat_duid.json'")
    break
else:
    print("Pilihan tidak valid. Masukkan angka 1-8.")

if __name__ == "__main__":
    main()
```